

AI24300/CS24300 PROJECT-I

**Secure Cloud-Based File Storage and
Document Management System
SRS + SDS REPORT**

BY

Ravinder Dhayal — BTECH/25188/24

Utkarsh Tikkiwal — BTECH/25190/24

Naman Kumar — BTECH/25170/24

Under the Supervision of

Dr. Kuntal Mukherjee

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

BIRLA INSTITUTE OF TECHNOLOGY, MESRA

JAIPUR CAMPUS, JAIPUR

MO-2026

Table of Contents

1. Introduction
 - 1.1 Purpose
 - 1.2 Scope
 - 1.3 Motivation
 - 1.4 Technology / Tools Used
2. Literature Review
 - 2.1 Cloud Storage Architecture
 - 2.2 Role-Based Access Control
 - 2.3 Secure File Sharing Mechanisms
 - 2.4 Identified Research Gap
3. Research Requirements (SRS Part)
 - 3.1 Functional Requirements
 - 3.2 Non-Functional Requirements
 - 3.3 Assumptions / Constraints
4. System Design (SDS Part)
 - 4.1 System Architecture
 - 4.2 Module Design
 - 4.3 Database Design
 - 4.4 API Design
 - 4.5 Security Design
5. Expected Outcomes
6. Semester-wise Workflow
7. References

1. Introduction

1.1 Purpose

The purpose of the Secure Cloud-Based File Storage and Document Management System is to develop a web-based platform that enables users to store, organize, share, and manage digital documents securely. The system is intended to provide an alternative to proprietary solutions like Google Drive and Dropbox by offering a self-hosted, privacy-focused file management platform with role-based access control, secure sharing mechanisms, and an intuitive user interface.

1.2 Scope

The scope of the project covers the design and development of a full-stack web application with the following capabilities:

- User registration, authentication, and session management.
- File and folder upload, download, rename, move, copy, and delete operations.
- Role-based access control for file sharing with configurable permissions.
- Shareable link generation with expiration dates and view/edit permissions.
- Search functionality for files and folders by name.
- Activity tracking and dashboard for monitoring file operations.
- Responsive web interface accessible across desktop, tablet, and mobile devices.
- Zero-cost deployment on free-tier cloud platforms.

1.3 Motivation

Modern organizations and individuals rely heavily on cloud-based file storage solutions. However, proprietary platforms raise concerns about data privacy, vendor lock-in, and recurring subscription costs. Existing self-hosted alternatives often lack modern UI/UX, secure sharing mechanisms, or proper access control. The motivation is to build a cost-effective, secure, and user-friendly file management system that ensures data ownership while demonstrating practical full-stack development skills using Java/Spring Boot and React.

1.4 Technology / Tools Used

Category	Technology / Tool	Purpose
Programming	Java 17, JavaScript/TypeScript	Backend and frontend implementation
Backend Framework	Spring Boot 3.x	REST API development and business logic
Frontend Framework	React.js 18+	User interface development
Styling	Tailwind CSS	Responsive and modern UI design
Database	PostgreSQL	Relational data storage
ORM	Spring Data JPA	Database operations and entity management
Security	Spring Security, JWT	Authentication and

		authorization
File Storage	Local filesystem	File upload and storage
API Client	Axios	HTTP communication between frontend and backend
Routing	React Router	Client-side navigation
Build Tools	Maven, npm	Dependency management and build automation
Version Control	Git, GitHub	Source code management and collaboration
Deployment	Vercel, Render, Supabase	Zero-cost cloud hosting

2. Literature Review

2.1 Cloud Storage Architecture

Ghemawat et al. introduced the Google File System (GFS), a scalable distributed file system designed for large data-intensive applications. The architecture separates metadata management from data storage, enabling efficient file operations at scale. This informs our design of separating file metadata (stored in PostgreSQL) from actual file content (stored on disk). [1]

2.2 Role-Based Access Control

Sandhu et al. established the NIST standard for Role-Based Access Control (RBAC), defining core RBAC, hierarchical RBAC, and constrained RBAC models. The principle of least privilege and separation of duties described in this work directly guides our permission system design for file sharing and folder access control. [2]

2.3 Secure File Sharing Mechanisms

Recent research on serverless file-sharing architectures demonstrates how encrypted URLs and key-value storage can achieve scalable and private file distribution without direct hosting. The approach of generating time-limited, permission-bound shareable links aligns with our sharing mechanism design. [3]

2.4 Identified Research Gap

The literature supports individual capabilities such as distributed file storage, RBAC, and secure sharing. However, there is a gap in integrating these capabilities into a single, cost-effective, self-hosted platform with modern UI/UX that can be deployed entirely on free-tier cloud services. Existing solutions either require expensive infrastructure or compromise on security and usability. This project addresses that gap by building a full-stack system with zero deployment cost.

3. Research Requirements (SRS Part)

3.1 Functional Requirements

The following functional requirements define the expected behavior of the proposed system. Each requirement has a unique identifier for traceability.

ID	Requirement	Statement
FR-01	User Registration	The system shall allow new users to register with email, password, and basic profile information.
FR-02	User Authentication	The system shall authenticate users via JWT-based login and support OAuth2 social login (Google).
FR-03	Session Management	The system shall maintain secure user sessions with token refresh and automatic expiration.
FR-04	File Upload	The system shall allow authenticated users to upload files with size validation and type checking.
FR-05	File Download	The system shall allow file owners and authorized users to download stored files securely.
FR-06	Folder Creation	The system shall support hierarchical folder creation, renaming, and organization.
FR-07	File Operations	The system shall support file rename, move, copy, and delete operations within the folder hierarchy.
FR-08	Drag-and-Drop	The system shall provide drag-and-drop interface for file upload and folder organization.
FR-09	Shareable Links	The system shall generate unique shareable links for files and folders with configurable permissions.
FR-10	Permission Control	The system shall support view-only and edit permissions on shared links with expiration dates.
FR-11	RBAC	The system shall implement

		role-based access control distinguishing between file owners, collaborators, and viewers.
FR-12	Search	The system shall provide search functionality for files and folders by name with filtering options.
FR-13	Activity Log	The system shall record and display user activities including uploads, downloads, shares, and modifications.
FR-14	Dashboard	The system shall present a user dashboard showing recent files, storage usage, and activity summary.
FR-15	Responsive UI	The system shall provide a responsive interface functional across desktop, tablet, and mobile devices.
FR-16	File Preview	The system shall support preview of common document types (PDF, images, text) without download.
FR-17	Trash/Recycle	The system shall provide a trash folder for deleted files with recovery capability.
FR-18	Storage Quota	The system shall enforce per-user storage quotas with warnings when approaching limits.
FR-19	API Documentation	The system shall expose REST APIs with Swagger/OpenAPI documentation.
FR-20	Deployment	The system shall be deployable on free-tier cloud platforms with HTTPS security.

3.2 Non-Functional Requirements

ID	Attribute	Requirement
NFR-01	Performance	The system shall respond to file operations within 2 seconds under normal load on student-level hardware.
NFR-02	Scalability	The architecture shall support increasing user

		count and file volume without redesign.
NFR-03	Security	All data in transit shall use HTTPS/TLS. Passwords shall be hashed using bcrypt.
NFR-04	Reliability	The system shall handle invalid inputs gracefully without crashes or data corruption.
NFR-05	Usability	The interface shall be intuitive with clear navigation and feedback for all operations.
NFR-06	Maintainability	Components shall be modular so that storage backend, auth method, or UI can be replaced independently.
NFR-07	Portability	The system shall run on standard Java and Node.js environments without proprietary dependencies.
NFR-08	Availability	The deployed prototype shall be accessible 24/7 on free-tier hosting with acceptable cold-start latency.
NFR-09	Resource Awareness	The implementation shall account for memory, storage, and API rate limits of free-tier services.
NFR-10	Reproducibility	The same codebase and configuration shall produce consistent deployment results.

3.3 Assumptions / Constraints

- The selected cloud platforms (Vercel, Render, Supabase) maintain their free-tier offerings throughout the project duration.
- Users have modern web browsers supporting HTML5, ES6+, and standard security protocols.
- File uploads are limited to common document types (PDF, DOCX, images, text) with a maximum size of 50MB per file.
- The system is a prototype and not intended for enterprise-scale production use.
- OAuth2 providers (Google) maintain their free authentication APIs.
- Storage is limited to local filesystem during development; cloud object storage is optional.

4. System Design (SDS Part)

4.1 System Architecture

The system follows a three-tier architecture: Presentation Layer (React frontend), Application Layer (Spring Boot backend), and Data Layer (PostgreSQL + local filesystem). The frontend communicates with the backend via REST APIs over HTTPS. The backend handles business logic, authentication, file operations, and database interactions.

- Presentation Layer: React.js application with Tailwind CSS, handling UI rendering, state management, and API calls.
- Application Layer: Spring Boot REST API with Spring Security, handling authentication, authorization, file operations, and business logic.
- Data Layer: PostgreSQL for metadata (users, files, folders, permissions, activities) and local filesystem for actual file storage.

4.2 Module Design

ID	Module	Primary Responsibility
M1	Authentication Module	User registration, login, JWT token generation, OAuth2 integration, session management.
M2	File Management Module	File upload, download, rename, move, copy, delete, and drag-and-drop handling.
M3	Folder Management Module	Hierarchical folder creation, organization, navigation, and tree structure maintenance.
M4	Sharing Module	Shareable link generation, permission assignment (view/edit), expiration date handling.
M5	RBAC Module	Role definition, permission checking, access control enforcement for all file operations.
M6	Search Module	File and folder name search with filtering, sorting, and result ranking.
M7	Activity Module	Logging user actions (upload, download, share, delete) and generating activity reports.
M8	Dashboard Module	User overview, recent files, storage usage statistics, and quick action interface.
M9	Trash Module	Soft delete handling, trash folder management, and file

		recovery functionality.
M10	API Module	REST endpoint definitions, request validation, response formatting, and Swagger documentation.

4.3 Database Design

The system uses PostgreSQL with the following core entities:

Entity	Attributes	Relationships	Purpose
User	id, email, password_hash, name, created_at, storage_quota, used_storage	One-to-Many with File, Folder, Activity	Stores user accounts and profile data
File	id, name, path, size, type, owner_id, folder_id, created_at, updated_at	Many-to-One with User, Folder	Stores file metadata and storage location
Folder	id, name, parent_id, owner_id, created_at, updated_at	Many-to-One with User, self-referential for hierarchy	Stores folder structure and hierarchy
ShareLink	id, token, file_id/folder_id, permission, expires_at, created_by	Many-to-One with User, File/Folder	Stores shareable link configurations
Activity	id, user_id, action, target_type, target_id, timestamp	Many-to-One with User	Logs all user actions for audit trail
Permission	id, user_id, file_id/folder_id, role (owner/viewer/editor)	Many-to-One with User, File/Folder	Stores explicit permission grants

4.4 API Design

The backend exposes RESTful APIs organized by module:

Endpoint	Method	Auth	Description
/api/auth/register	POST	No	User registration
/api/auth/login	POST	No	JWT token generation
/api/auth/oauth/google	GET	No	Google OAuth2 callback
/api/files/upload	POST	Yes	File upload with metadata
/api/files/{id}/download	GET	Yes	File download
/api/files/{id}	DELETE	Yes	File deletion (soft delete)

/api/files/{id}/rename	PUT	Yes	File rename
/api/files/{id}/move	PUT	Yes	Move file to different folder
/api/folders	POST	Yes	Create new folder
/api/folders/{id}/contents	GET	Yes	List folder contents
/api/share	POST	Yes	Generate shareable link
/api/share/{token}	GET	No/Yes	Access shared resource
/api/search	GET	Yes	Search files and folders
/api/activity	GET	Yes	Get user activity log
/api/dashboard	GET	Yes	Get dashboard statistics

4.5 Security Design

- Password Hashing: User passwords are hashed using bcrypt with salt before storage.
- JWT Authentication: Stateless authentication using signed JWT tokens with expiration.
- HTTPS/TLS: All API communications encrypted in transit.
- RBAC Enforcement: Every file operation checks user permissions before execution.
- Input Validation: All API inputs validated for type, size, and format to prevent injection attacks.
- CORS Configuration: Cross-origin requests restricted to approved frontend domains.
- Rate Limiting: API endpoints protected against abuse with request rate limits.

5. Expected Outcomes

- A working prototype of a secure cloud-based file storage and document management system.
- User authentication with JWT and OAuth2 social login support.
- Intuitive file and folder management with drag-and-drop interface.
- Secure sharing mechanism with configurable permissions and expiration dates.
- Role-based access control enforcing principle of least privilege.
- Search functionality for quick file and folder discovery.
- Activity tracking and dashboard for monitoring user actions.
- Responsive web interface functional across all device types.
- Zero-cost deployment on free-tier cloud platforms with HTTPS.
- Comprehensive API documentation using Swagger/OpenAPI.

6. Semester-wise Workflow

Semester	Planned Work
Semester 5	Literature review and requirement analysis; database schema design; Spring Boot project setup; user authentication module (JWT + OAuth2); basic file upload/download API; React project setup and login UI.
Semester 6	File and folder management modules; drag-and-drop interface; RBAC implementation; shareable link generation; search functionality; activity logging; dashboard UI; integration testing.
Semester 7	Trash/recycle functionality; file preview; storage quota enforcement; responsive design refinement; deployment on Vercel/Render/Supabase; security hardening; final testing; documentation and project report.

7. References

- [1] S. Ghemawat, H. Gobioff, and S.-T. Leung, "The Google File System," in Proc. 19th ACM SOSP, Bolton Landing, NY, USA, 2003, pp. 29-43.
- [2] R. Sandhu, D. Ferraiolo, and R. Kuhn, "The NIST Model for Role-Based Access Control: Towards a Unified Standard," in Proc. 5th ACM Workshop on Role-Based Access Control, Berlin, Germany, 2000, pp. 47-63.
- [3] "Scalable anonymity in file sharing: Leveraging Serverless Technologies for Secure and Private File Distribution," in Proc. IEEE Conf., Bali, Indonesia, Jun. 2024, pp. 1-6.
- [4] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," Advances in Neural Information Processing Systems, vol. 33, 2020.

End of SRS + SDS Report